

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Recap

- Vorstellungen Aufgaben
- Q&A Dokumente, um wieder in die Themen reinzukommen
 - OOP_Cpp_Part_7_Q&A

Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Kapitel 5: Clean Code



51. Motivation

52. Clean Code – Robert C. Martin

53. Weitere Paradigmen

54. Debugging

Was ist Clean Code?

Clean Code ist einfach verständlicher, gut zu wartender und leicht zu erweiternder Code.

- Was genau das heißt ist sehr individuell
- Clean Code ist auch **Synonym für eine Sammlung an Good Practices und Konventionen** (kein striktes Regelwerk)
- Begriff geht zurück auf gleichnamiges Buch von Robert C. Martin (später mehr)

Warum Clean Code?



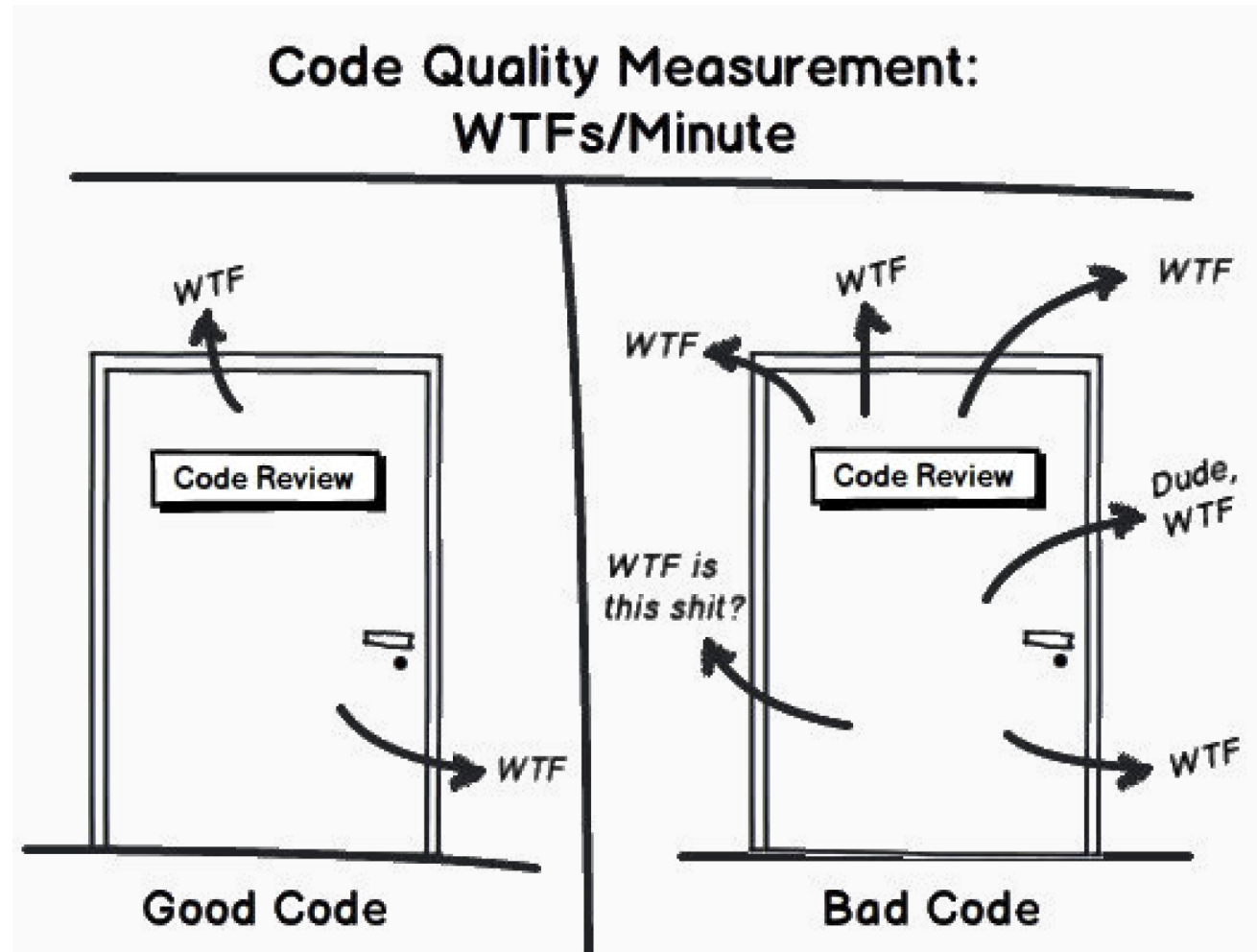
Code Lesen

Was macht dieser Code?

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS
• * Executing task: C:/Windows/System32/cmd.exe /d /c g++ -Wall -Wextra -Wpedantic -Wshadow -W
ntro\sh.cpp -o .\build\Debug\sh.o && g++ -Wall -Wextra -Wpedantic -Wshadow -Wformat=2 -Wcast-a
* Terminal will be reused by tasks, press any key to close it.
• * Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe
Enter s: 12
Square A: 22.32
Enter b: 20
Enter h: 5
Triangle A: 7.75
* Terminal will be reused by tasks, press any key to close it.
```

```
01_lecture > 5_Clean_Code > 51_intro > C++ sh.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  class sh {
5  public:
6      double f1(double l) {
7          double q_area = 0.155 * l * l;
8          return q_area;
9      }
10     double f2(double bb, double* hh) {
11         double t_area = 0.0775 * bb * (*hh);
12         return t_area;
13     }
14 private:
15     double tmp;
16 };
17
18 int main() {
19     double qside, trbase, trheight, qresult, tresult;
20     cout << "Enter s: ";
21     cin >> qside;
22     sh s;
23     qresult = s.f1(qside);
24     cout << "Square A: " << qresult << endl;
25
26     cout << "Enter b: ";
27     cin >> trbase;
28     cout << "Enter h: ";
29     cin >> trheight;
30     tresult = s.f2(trbase, &trheight);
31     cout << "Triangle A: " << tresult << endl;
32
33
34     return 0;
35 }
```

Die einzig echte Messgröße für guten Code...



Code Refactoring

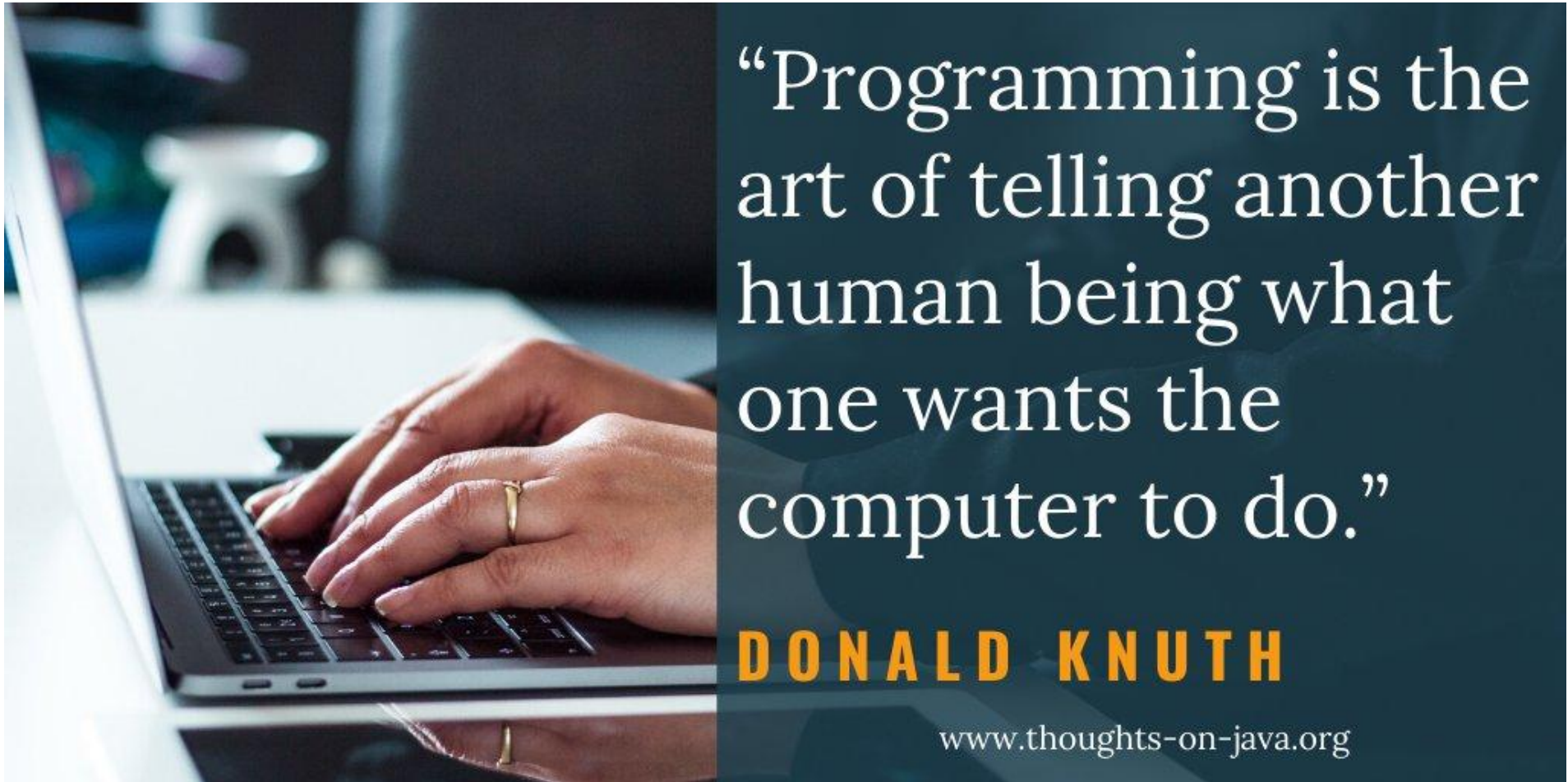
Wie viele WTFs hatten Sie in den letzten Minuten? Wir versuchen weniger wtf's zu haben →

```
6_Clean_Code > 51_intro > C++ sh_refactored_OOP.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <memory>
4
5  // Konstanten
6
7  // Abstrakte Basisklasse
8
9  // Abgeleitete Klassen
10
11 // Funktionen
12
13 // Hauptfunktion zur Eingabe der Formen und Ausgabe der Flächeninformationen
14 void getInputAndPrintAreas()
15 {
16     auto shapes = getShapesFromUser(); // Erstellen der Formen basierend auf Benutzereingaben
17     printShapeAreas(shapes);           // Ausgabe der Flächeninformationen   identifier "prints"
18 }
19
20 int main()
21 {
22     getInputAndPrintAreas(); // Aufrufen der Hauptfunktion
23     return 0;
24 }
```

Code Refactoring

```
01_lecture > 5_Clean_Code > 51_intro > C++ sh_refactored_OOP.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <memory>
4
5  constexpr double SQ_CM_TO_SQ_INCH = 0.155; // Konstante zur Umrechnung von Quadratcentimetern in Quadratzoll
6
7  // Abstrakte Basisklasse für Formen
8 > class Shape ...
19
20 // Klasse für Quadrate, abgeleitet von Shape
21 > class Square : public Shape ...
37
38 // Klasse für Dreiecke, abgeleitet von Shape
39 > class Triangle : public Shape ...
56
57 // Funktion zur Eingabe der Seitenlänge eines Quadrats
58 > double getSquareSide() ...
65
66 // Funktion zur Eingabe der Basis und Höhe eines Dreiecks
67 > std::pair<double, double> getTriangleDimensions() ...
76
77 // Funktion zur Erstellung der Formen basierend auf Benutzereingaben
78 > std::vector<std::unique_ptr<Shape>> getShapesFromUser() ...
89
90 // Funktion zur Ausgabe der Flächeninformationen aller Formen
91 > void printShapeAreas(const std::vector<std::unique_ptr<Shape>> &shapes) ...
98
99 // Hauptfunktion zur Eingabe der Formen und Ausgabe der Flächeninformationen
100 > void getInputAndPrintAreas() ...
105
106 int main()
107 {
108     getInputAndPrintAreas(); // Aufrufen der Hauptfunktion
109     return 0;
110 }
```

Schlechte Code Qualität fährt Projekte gegen die Wand!



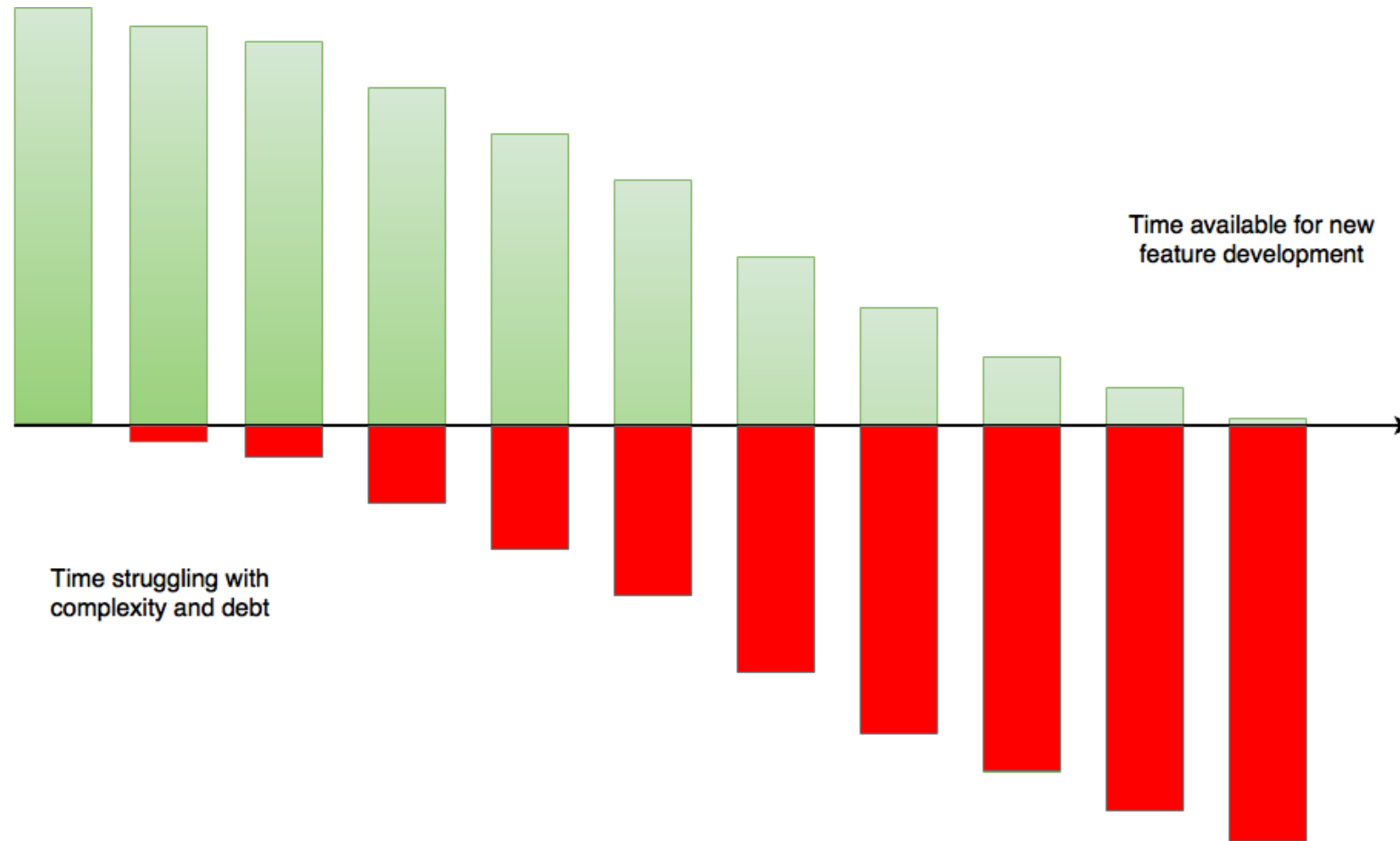
Softwareentwicklung: Lesen statt schreiben!

- Sie werden in Ihrem Leben mehr Zeit damit verbringen, Code zu lesen, als Code zu schreiben ... viel mehr. 
- Die Verteilung von Code lesen zu Code Schreiben ist in der professionellen Software Entwicklung:

> 90% zu < 10%

Deshalb sollte die oberste Priorität beim geschriebenen Code auf der Lesbarkeit (und Wartbarkeit) liegen!

Technische Schuld: *Technical debt*



Kapitel 5: Clean Code

51. Motivation



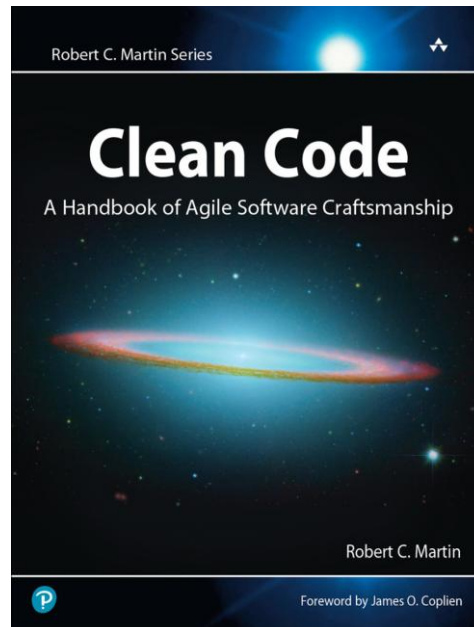
52. Clean Code – Robert C. Martin

53. Weitere Paradigmen

54. Debugging

Das Buch aus dem ich am meisten gelernt habe ...

- **“Clean Code: A Handbook of Agile Software Craftmanship”** - Robert C. Martin



(... über die professionelle Softwareentwicklung)

Was ist Clean Code?

Bjarne Stroustrup, inventor of C++ and author of *The C++ Programming Language*

I like my code to be elegant and efficient. The logic should be straightforward to make it hard for bugs to hide, the dependencies minimal to ease maintenance, error handling complete according to an articulated strategy, and performance close to optimal so as not to tempt people to make the code messy with unprincipled optimizations. Clean code does one thing well.



“Big” Dave Thomas, founder of OTI, godfather of the Eclipse strategy

Clean code can be read, and enhanced by a developer other than its original author. It has unit and acceptance tests. It has meaningful names. It provides one way rather than many ways for doing one thing. It has minimal dependencies, which are explicitly defined, and provides a clear and minimal API. Code should be literate since depending on the language, not all necessary information can be expressed clearly in code alone.



Ron Jeffries, author of *Extreme Programming Installed* and *Extreme Programming Adventures in C#*

Ron began his career programming in Fortran at the Strategic Air Command and has written code in almost every language and on almost every machine. It pays to consider his words carefully.

In recent years I begin, and nearly end, with Beck's rules of simple code. In priority order, simple code:

- Runs all the tests;
- Contains no duplication;
- Expresses all the design ideas that are in the system;
- Minimizes the number of entities such as classes, methods, functions, and the like.

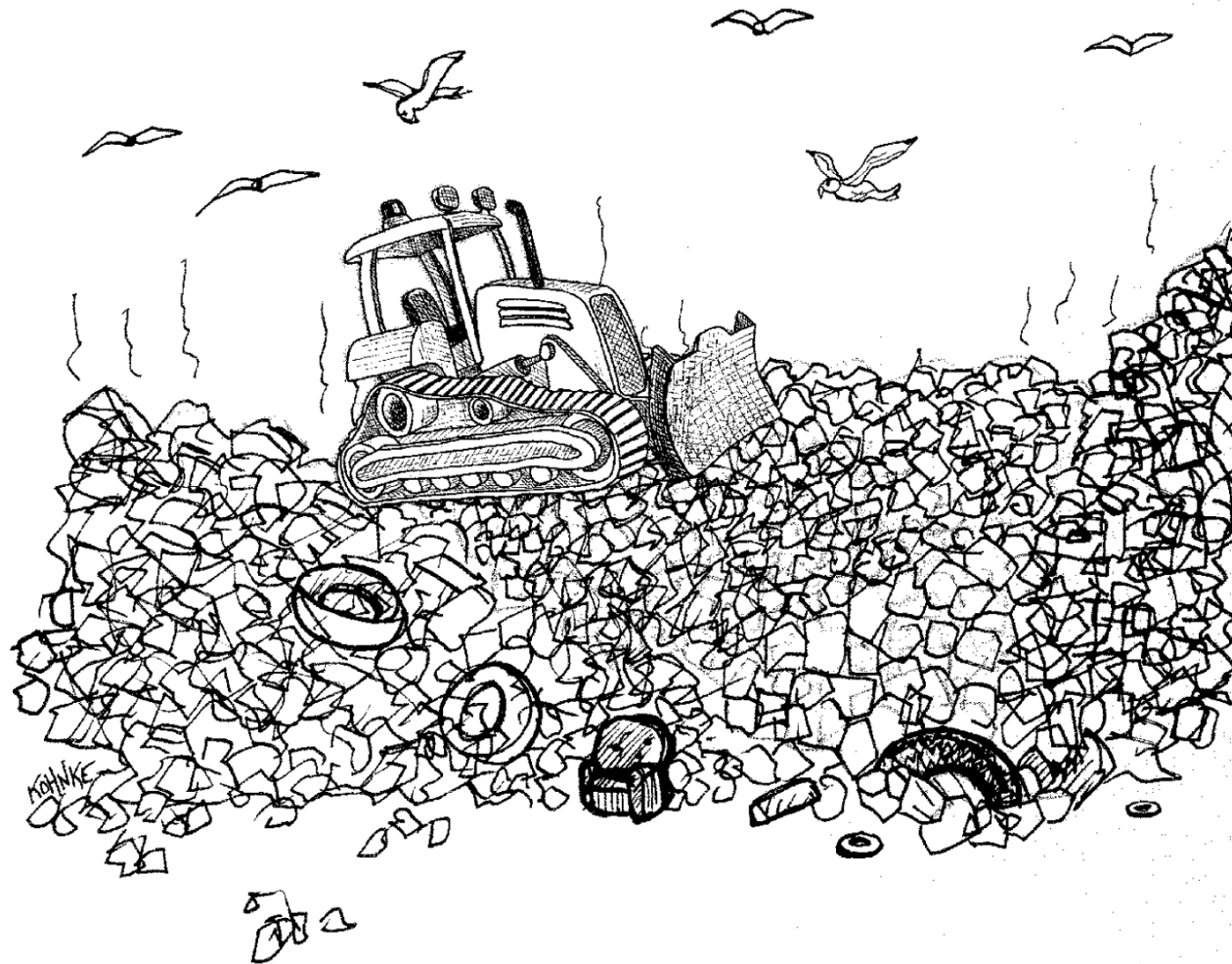


Grady Booch, author of *Object Oriented Analysis and Design with Applications*

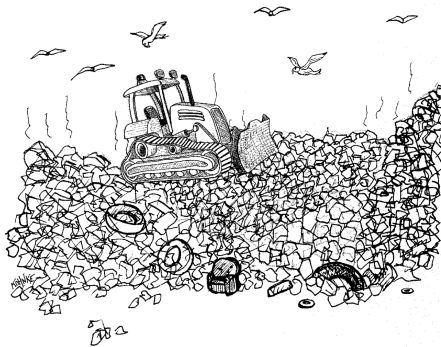
Clean code is simple and direct. Clean code reads like well-written prose. Clean code never obscures the designer's intent but rather is full of crisp abstractions and straightforward lines of control.



Code Smells



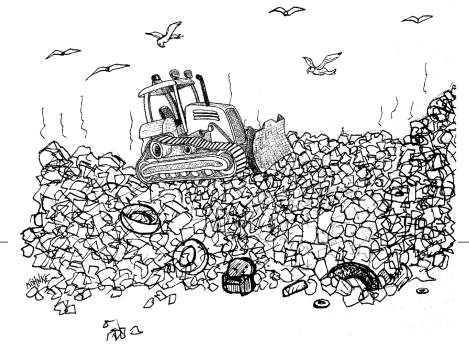
Clean Code: Smells and Heuristics



Chapter 17: Smells and Heuristics	285		
Comments	286		
C1: Inappropriate Information	286		
C2: Obsolete Comment	286		
C3: Redundant Comment	286		
C4: Poorly Written Comment	287		
C5: Commented-Out Code	287		
Environment	287		
E1: Build Requires More Than One Step	287		
E2: Tests Require More Than One Step	287		
Functions	288		
F1: Too Many Arguments	288		
F2: Output Arguments	288		
F3: Flag Arguments	288		
F4: Dead Function	288		
General	288		
G1: Multiple Languages in One Source File	288		
G2: Obvious Behavior Is Unimplemented	288		
G3: Incorrect Behavior at the Boundaries	289		
G4: Overridden Safeties	289		
G5: Duplication	289		
G6: Code at Wrong Level of Abstraction	290		
G7: Base Classes Depending on Their Derivatives	291		
G8: Too Much Information	291		
G9: Dead Code	292		
G10: Vertical Separation	292		
G11: Inconsistency	292		
G12: Clutter	293		
G13: Artificial Coupling	293		
G14: Feature Envy	293		
G15: Selector Arguments	294		
G16: Obscured Intent	295		
G17: Misplaced Responsibility	295		
G18: Inappropriate Static	296		
G19: Use Explanatory Variables	296		
G20: Function Names Should Say What They Do	297		
G21: Understand the Algorithm	297		
G22: Make Logical Dependencies Physical	298		
G23: Prefer Polymorphism to If/Else or Switch/Case	299		
G24: Follow Standard Conventions	299		
G25: Replace Magic Numbers with Named Constants	300		
G26: Be Precise	301		
G27: Structure over Convention	301		
G28: Encapsulate Conditionals	301		
G29: Avoid Negative Conditionals	302		
G30: Functions Should Do One Thing	302		
G31: Hidden Temporal Couplings	302		
G32: Don't Be Arbitrary	303		
G33: Encapsulate Boundary Conditions	304		
G34: Functions Should Descend Only One Level of Abstraction	304		
G35: Keep Configurable Data at High Levels	306		
G36: Avoid Transitive Navigation	306		
Names	309		
N1: Choose Descriptive Names	309		
N2: Choose Names at the Appropriate Level of Abstraction	311		
N3: Use Standard Nomenclature Where Possible	311		
N4: Unambiguous Names	312		
N5: Use Long Names for Long Scopes	312		
N6: Avoid Encodings	312		
N7: Names Should Describe Side-Effects	313		
Tests	313		
T1: Insufficient Tests	313		
T2: Use a Coverage Tool!	313		
T3: Don't Skip Trivial Tests	313		
T4: An Ignored Test Is a Question about an Ambiguity	313		
T5: Test Boundary Conditions	314		
T6: Exhaustively Test Near Bugs	314		
T7: Patterns of Failure Are Revealing	314		
T8: Test Coverage Patterns Can Be Revealing	314		
T9: Tests Should Be Fast	314		

Der folgende Auszug soll Ihnen dabei helfen, Ihren Projektcode besser zu verstehen und mit Ihrem Teampartner eindeutig zu teilen.
Für die Bewertung sind diese Punkte ebenfalls ein sehr guter Hinweis.

Clean Code: Smells and Heuristics



Chapter 17: Smells and Heuristics	285
Comments	286
C1: <i>Inappropriate Information</i>	286
C2: <i>Obsolete Comment</i>	286
C3: <i>Redundant Comment</i>	286
C4: <i>Poorly Written Comment</i>	287
C5: <i>Commented-Out Code</i>	287
Environment	287
E1: <i>Build Requires More Than One Step</i>	287
E2: <i>Tests Require More Than One Step</i>	287
Functions	288
F1: <i>Too Many Arguments</i>	288
F2: <i>Output Arguments</i>	288
F3: <i>Flag Arguments</i>	288
F4: <i>Dead Function</i>	288
General	288
G1: <i>Multiple Languages in One Source File</i>	288
G2: <i>Obvious Behavior Is Unimplemented</i>	288
G3: <i>Incorrect Behavior at the Boundaries</i>	289
G4: <i>Overridden Safeties</i>	289
G5: <i>Duplication</i>	289
G6: <i>Code at Wrong Level of Abstraction</i>	290
G7: <i>Base Classes Depending on Their Derivatives</i>	291
G8: <i>Too Much Information</i>	291
G9: <i>Dead Code</i>	292
G10: <i>Vertical Separation</i>	292
G11: <i>Inconsistency</i>	292
G12: <i>Clutter</i>	293

G13: <i>Artificial Coupling</i>	293
G14: <i>Feature Envy</i>	293
G15: <i>Selector Arguments</i>	294
G16: <i>Obscured Intent</i>	295
G17: <i>Misplaced Responsibility</i>	295
G18: <i>Inappropriate Static</i>	296
G19: <i>Use Explanatory Variables</i>	296
G20: <i>Function Names Should Say What They Do</i>	297
G21: <i>Understand the Algorithm</i>	297
G22: <i>Make Logical Dependencies Physical</i>	298
G23: <i>Prefer Polymorphism to If/Else or Switch/Case</i>	299
G24: <i>Follow Standard Conventions</i>	299
G25: <i>Replace Magic Numbers with Named Constants</i>	300
G26: <i>Be Precise</i>	301
G27: <i>Structure over Convention</i>	301
G28: <i>Encapsulate Conditionals</i>	301
G29: <i>Avoid Negative Conditionals</i>	302
G30: <i>Functions Should Do One Thing</i>	302
G31: <i>Hidden Temporal Couplings</i>	302
G32: <i>Don't Be Arbitrary</i>	303
G33: <i>Encapsulate Boundary Conditions</i>	304
G34: <i>Functions Should Descend Only One Level of Abstraction</i>	304
G35: <i>Keep Configurable Data at High Levels</i>	306
G36: <i>Avoid Transitive Navigation</i>	306

Names	309
N1: <i>Choose Descriptive Names</i>	309
N2: <i>Choose Names at the Appropriate Level of Abstraction</i>	311
N3: <i>Use Standard Nomenclature Where Possible</i>	311
N4: <i>Unambiguous Names</i>	312
N5: <i>Use Long Names for Long Scopes</i>	312
N6: <i>Avoid Encodings</i>	312
N7: <i>Names Should Describe Side-Effects</i>	313

xvi

Contents

Tests	313
T1: <i>Insufficient Tests</i>	313
T2: <i>Use a Coverage Tool!</i>	313
T3: <i>Don't Skip Trivial Tests</i>	313
T4: <i>An Ignored Test Is a Question about an Ambiguity</i>	313
T5: <i>Test Boundary Conditions</i>	314
T6: <i>Exhaustively Test Near Bugs</i>	314
T7: <i>Patterns of Failure Are Revealing</i>	314
T8: <i>Test Coverage Patterns Can Be Revealing</i>	314
T9: <i>Tests Should Be Fast</i>	314

Kommentare

- **C1 Inappropriate Information:** Kommentare sollten technischen Anmerkungen zum Code und Design vorbehalten sein.
 - Change History, Autor etc. → Git (Sourcecode-Control-System)
- **C2 Obsolete Comment:** Veraltete Kommentare sind schlechter als keine Kommentare!!!
- **C3 Redundant Comment :** Redundante Kommentare vermeiden
 - Kommentare sollten Dinge ausdrücken, die der Code selbst nicht mitteilen kann. `i++; // increment i` 
- **C5 Commented-Out Code:** Auskommentierter Code sollte gelöscht werden !!!
 - Wenn Sie auskommentierten Code sehen, löschen Sie ihn! Keine Bange!
 - Das Sourcecode-Control-System erinnert sich an den Code.

Kommentare

Welche Code-Smells können Sie hier erkennen?

```
// Author: Max Mustermann  
// Created: 2018-05-10  
// Modified: 2021-09-14  
int calculateArea(int width, int height) {  
    // Berechnet die Fläche eines Quadrats mit der Seitenlänge 'length'  
    // returnValue = width * height;  
    return width * height; // Breite * Höhe  
}
```

Funktionen

- Funktionslänge: **so klein wie möglich**
 - Persönliche Präferenz: Funktionen sollten kaum länger als 20 Zeilen sein (besonders wenn man sich streng an SRP halten sollte)
https://en.wikipedia.org/wiki/Single-responsibility_principle
- **F1 Too Many Arguments:** Funktionen sollten eine kleine Anzahl von Argumenten haben
 - Keine sind am besten, dann 2, 3
 - Für alles über 3 sollte man eine sehr, sehr gute Begründung haben.
- **F2 Output Arguments:** Output Arguments vermeiden, Funktionen sollten immer nur eine Aufgabe haben.
- **F4 Dead Function:** Tote Funktionen löschen (nicht auskommentieren!)

```
int divide(int dividend, int divisor, int &remainder) {  
    if (divisor == 0) {  
        throw std::invalid_argument("Division by zero");  
    }  
    int quotient = dividend / divisor;  
    remainder = dividend % divisor;  
    return quotient;  
}
```

Funktionen

Welche Code-Smells können Sie hier erkennen?

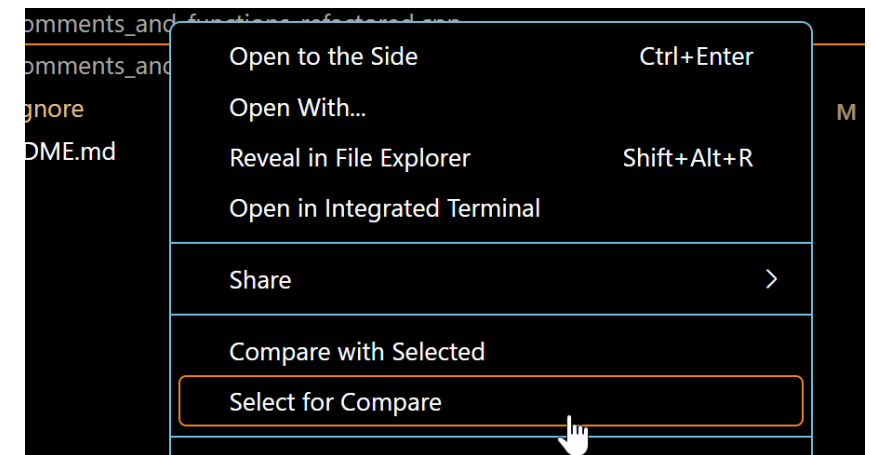
```
void logToConsole(const std::string& msg) {
    std::cout << "LOG: " << msg << std::endl;
}

void processData(
    const std::string& name,
    int age,
    const std::string& email,
    bool isAdmin,
    int& statusCode,
    std::string& welcomeMsg
) {
    if (name.empty() || email.empty()) {
        statusCode = -1;
        welcomeMsg = "Invalid data.";
        return;
    }

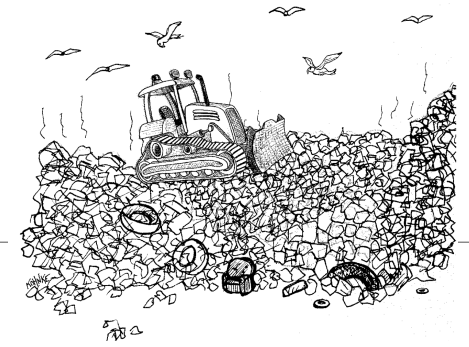
    statusCode = 0;
    welcomeMsg = "Welcome, " + name + (isAdmin ? " (Admin)" : "");
}
```


Aufgaben: Refactoring Kommentare & Funktionen

1. Laden Sie sich “comments_and_functions_smelly.cpp” vom Git und erstellen Sie eine Kopie der Datei zur Bearbeitung
2. Führen Sie ein Refactoring für Kommentare und Funktionen durch
3. Zeigen Sie die Änderungen mit dem “Compare” Feature



Clean Code: Smells and Heuristics



Chapter 17: Smells and Heuristics	285
Comments	286
C1: <i>Inappropriate Information</i>	286
C2: <i>Obsolete Comment</i>	286
C3: <i>Redundant Comment</i>	286
C4: <i>Poorly Written Comment</i>	287
C5: <i>Commented-Out Code</i>	287
Environment	287
E1: <i>Build Requires More Than One Step</i>	287
E2: <i>Tests Require More Than One Step</i>	287
Functions	288
F1: <i>Too Many Arguments</i>	288
F2: <i>Output Arguments</i>	288
F3: <i>Flag Arguments</i>	288
F4: <i>Dead Function</i>	288
General	288
G1: <i>Multiple Languages in One Source File</i>	288
G2: <i>Obvious Behavior Is Unimplemented</i>	288
G3: <i>Incorrect Behavior at the Boundaries</i>	289
G4: <i>Overridden Safeties</i>	289
G5: <i>Duplication</i>	289
G6: <i>Code at Wrong Level of Abstraction</i>	290
G7: <i>Base Classes Depending on Their Derivatives</i>	291
G8: <i>Too Much Information</i>	291
G9: <i>Dead Code</i>	292
G10: <i>Vertical Separation</i>	292
G11: <i>Inconsistency</i>	292
G12: <i>Chatter</i>	293

G13: <i>Artificial Coupling</i>	293
G14: <i>Feature Envy</i>	293
G15: <i>Selector Arguments</i>	294
G16: <i>Obscured Intent</i>	295
G17: <i>Misplaced Responsibility</i>	295
G18: <i>Inappropriate Static</i>	296
G19: <i>Use Explanatory Variables</i>	296
G20: <i>Function Names Should Say What They Do</i>	297
G21: <i>Understand the Algorithm</i>	297
G22: <i>Make Logical Dependencies Physical</i>	298
G23: <i>Prefer Polymorphism to If/Else or Switch/Case</i>	299
G24: <i>Follow Standard Conventions</i>	299
G25: <i>Replace Magic Numbers with Named Constants</i>	300
G26: <i>Be Precise</i>	301
G27: <i>Structure over Convention</i>	301
G28: <i>Encapsulate Conditionals</i>	301
G29: <i>Avoid Negative Conditionals</i>	302
G30: <i>Functions Should Do One Thing</i>	302
G31: <i>Hidden Temporal Couplings</i>	302
G32: <i>Don't Be Arbitrary</i>	303
G33: <i>Encapsulate Boundary Conditions</i>	304
G34: <i>Functions Should Descend Only</i> <i>One Level of Abstraction</i>	304
G35: <i>Keep Configurable Data at High Levels</i>	306
G36: <i>Avoid Transitive Navigation</i>	306

Names	309
N1: <i>Choose Descriptive Names</i>	309
N2: <i>Choose Names at the Appropriate Level of Abstraction</i>	311
N3: <i>Use Standard Nomenclature Where Possible</i>	311
N4: <i>Unambiguous Names</i>	312
N5: <i>Use Long Names for Long Scopes</i>	312
N6: <i>Avoid Encodings</i>	312
N7: <i>Names Should Describe Side-Effects</i>	313

xvi

Contents

Tests	313
T1: <i>Insufficient Tests</i>	313
T2: <i>Use a Coverage Tool!</i>	313
T3: <i>Don't Skip Trivial Tests</i>	313
T4: <i>An Ignored Test Is a Question about an Ambiguity</i>	313
T5: <i>Test Boundary Conditions</i>	314
T6: <i>Exhaustively Test Near Bugs</i>	314
T7: <i>Patterns of Failure Are Revealing</i>	314
T8: <i>Test Coverage Patterns Can Be Revealing</i>	314
T9: <i>Tests Should Be Fast</i>	314

General

- **G3 Incorrect Behaviour at the Boundaries:** Testen Sie alle Grenz- und Sonderfälle.
 - Es gibt keinen Ersatz für die gebotene Sorgfalt. Grenzbedingungen, Sonderfälle, Eigenheiten und Ausnahmen können einen eleganten und intuitiven Algorithmus verunstalten.
 - **Verlassen Sie sich nicht auf Ihre Intuition.**
- **G5 Duplication:** DRY: *Don't repeat yourself!*
Eine der wichtigsten Regeln: „*Once, and only once.*“
 - Wiederholung stellen Sie vor Probleme:
 - Verpasste Chance von Abstraktion
 - Doppelte Wartung
 - Doppelter Optimierungsaufwand

General

- **G6 Code at Wrong Level of Abstraction:** Die richtige Abstraktions Ebene in Funktionen und Methoden finden
- **G8 Too Much Information:** Vermeiden, dass eine Funktion/Methode zu viel Information benötigt / hergibt
 - Interfaces klein halten.
 - Aufteilen auf viele kleine Funktionen/Klassen.
- **G9 Dead Code:** Vermeiden Sie toten Code
 - Body einer if-Anweisung, die eine Bedingung prüft, die nicht eintreten kann; im catch-Block einer try-Anweisung, die nie throws; in kleinen Utility-Methoden, die nie aufgerufen werden; oder in switch/case-Bedingungen, die niemals eintreten.
- **G10 Vertical Separation:** Vertikale Nähe Hilft der Lesbarkeit (oberstes Gebot in der SW-Entwicklung)
 - Definition von (lokalen) Variablen dort wo sie gebraucht werden.
 - Private Funktionen sollten direkt unter ihrer ersten Verwendung definiert werden.

General

- **G11 Inconsistency:** Vermeiden Sie Inkonsistenz (eines der wichtigsten Gebote)
 - Wenn Sie etwas in einer gewissen Weise tun, sollten Sie alle ähnlichen Aufgaben auf dieselbe Weise erledigen. Diese geht auf das Prinzip der geringsten Überraschung zurück. Wählen Sie Ihre Konventionen sorgfältig aus; und wenn Sie sie gewählt haben, wenden Sie sie sorgfältig an.
- **G13 Artificial Coupling:** Künstliche Kopplung vermeiden
 - Beispielsweise sollten allgemeine Enums nicht in speziellere Klassen eingeschlossen werden, weil dadurch die gesamte Anwendung gezwungen wird, diese spezielleren Klassen zu kennen.
 - Dasselbe gilt für statische Allzweckfunktionen, die in speziellen Klassen deklariert werden.

General

- **G16 Obscured Intend:** Code sollte sagen was er tut
 - beschreibende Namen (Suchbare Namen verwenden, vermeiden Sie Wortspiele)
 - keine Magic-Numbers
 - Nutzen Sie (korrekte) fachspezifische Sprache

```
// Schlecht
int accountList; // Ist das eine Liste? Oder vielleicht ein einzelnes Konto?

// Gut
int accountCount; // Klar und verständlich, dass es sich um eine Zählvariable handelt.
```

```
// Schlecht
int accNum; // Abkürzung könnte unklar sein

// Gut
int accountNumber;
```

```
// Funktion in CamelCase
void calculateInterest();

// Klasse in PascalCase
class BankAccount;
```

```
// Schlecht
int elapsed; // Was ist verstrichen? Zeit, Tage, Sekunden?

// Gut
int elapsedTimeInDays;
```

General

- **G17 Misplaced Responsibility:** Least-Surprise-Prinzip (Prinzip der geringsten Überraschung)
Wo sollte Code stehen (z.B. PI als Konstante in Math class, in Trigonometry class, in Constants?)
 - Code sollte dort eingefügt werden, wo ihn ein Leser natürlicherweise erwarten würde.
 - Die Konstante PI sollte dort stehen, wo die trigonometrischen Funktionen deklariert werden.
- **G18 Inappropriate Static:** *Free your functions!*
 - Wenn statische Funktionen auch frei sein können ist dies oft besser.
 - Noch präziser gesagt: nutzen Sie statische Funktionen nur wenn ausgeschlossen werden kann, dass die Funktion polymorph sein sollte.

General

- **G19 Use Explanatory Variables:** Variablen sollten sagen was sie sind (und Substantive sein).
- **G20 Function Names Should Say What They Do:** Funktionsnamen sollten sagen was sie machen (und Verben sein).
- **G25 Replace Magic Numbers with Named Constants:** Magic Numbers mit benannten Konstanten ersetzen.

```
int daysSinceLastLogin; // Beschreibt klar, dass es sich um eine Anzahl von Tagen handelt
double accountBalance; // Beschreibt klar den Kontostand
```

```
void calculateInterest(); // Klar, dass Zinsen berechnet werden
```

```
const int MAX_ITERATIONS = 10;
for (int i = 0; i < MAX_ITERATIONS; ++i) {
    // Code
}

const double DEFAULT_INTEREST_RATE = 0.05;
double interestRate = DEFAULT_INTEREST_RATE; // Klar definierte Konstante
```

General

- **G29 Avoid Negative Conditionals:**

```
if (buffer.shouldCompact())
```

is preferable to

```
if (!buffer.shouldNotCompact())
```

- **G30 Functions Should Do One Thing:** Funktionen sollten **eine** und auch nur **eine** Aufgabe haben!

```
public void pay() {  
    for (Employee e : employees) {  
        if (e.isPayday()) {  
            Money pay = e.calculatePay();  
            e.deliverPay(pay);  
        }  
    }  
}
```

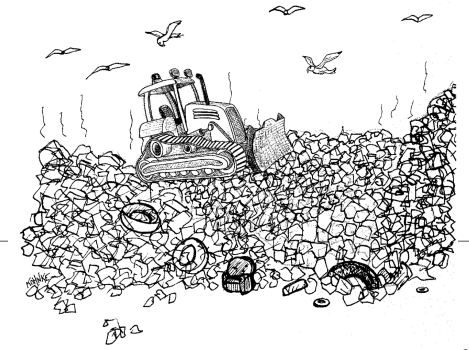


```
public void pay() {  
    for (Employee e : employees)  
        payIfNecessary(e);  
}  
  
private void payIfNecessary(Employee e) {  
    if (e.isPayday())  
        calculateAndDeliverPay(e);  
}  
  
private void calculateAndDeliverPay(Employee e) {  
    Money pay = e.calculatePay();  
    e.deliverPay(pay);  
}
```

General

- **G36 Avoid Transitive Navigation:** Law of Demeter: *Writing shy code*: Eine Klasse sollte nur von ihren direkten Nachbarn abhängen
 - **Nicht:** `a.getB().getC().doSomething();`
 - **Besser:** `myCollaborator.doSomething();`
 - Eine Methode eines Objekts sollte nur die Methoden folgender Objekte aufrufen:
 1. Das Objekt selbst.
 2. Objekte, die als Parameter an die Methode übergeben werden.
 3. Objekte, die innerhalb der Methode erstellt oder instanziiert werden.
 4. Direkte Komponenten des Objekts (d. h., Felder des Objekts).

Clean Code: Smells and Heuristics



Chapter 17: Smells and Heuristics	285
Comments	286
C1: Inappropriate Information	286
C2: Obsolete Comment	286
C3: Redundant Comment	286
C4: Poorly Written Comment	287
C5: Commented-Out Code	287
Environment	287
E1: Build Requires More Than One Step	287
E2: Tests Require More Than One Step	287
Functions	288
F1: Too Many Arguments	288
F2: Output Arguments	288
F3: Flag Arguments	288
F4: Dead Function	288
General	288
G1: Multiple Languages in One Source File	288
G2: Obvious Behavior Is Unimplemented	288
G3: Incorrect Behavior at the Boundaries	289
G4: Overridden Safeties	289
G5: Duplication	289
G6: Code at Wrong Level of Abstraction	290
G7: Base Classes Depending on Their Derivatives	291
G8: Too Much Information	291
G9: Dead Code	292
G10: Vertical Separation	292
G11: Inconsistency	292
G12: Chatter	293

G13: Artificial Coupling	293
G14: Feature Envy	293
G15: Selector Arguments	294
G16: Obscured Intent	295
G17: Misplaced Responsibility	295
G18: Inappropriate Static	296
G19: Use Explanatory Variables	296
G20: Function Names Should Say What They Do	297
G21: Understand the Algorithm	297
G22: Make Logical Dependencies Physical	298
G23: Prefer Polymorphism to If/Else or Switch/Case	299
G24: Follow Standard Conventions	299
G25: Replace Magic Numbers with Named Constants	300
G26: Be Precise	301
G27: Structure over Convention	301
G28: Encapsulate Conditionals	301
G29: Avoid Negative Conditionals	302
G30: Functions Should Do One Thing	302
G31: Hidden Temporal Couplings	302
G32: Don't Be Arbitrary	303
G33: Encapsulate Boundary Conditions	304
G34: Functions Should Descend Only One Level of Abstraction	304
G35: Keep Configurable Data at High Levels	306
G36: Avoid Transitive Navigation	306

Names	309
N1: Choose Descriptive Names	309
N2: Choose Names at the Appropriate Level of Abstraction	311
N3: Use Standard Nomenclature Where Possible	311
N4: Unambiguous Names	312
N5: Use Long Names for Long Scopes	312
N6: Avoid Encodings	312
N7: Names Should Describe Side-Effects.	313

xvi

Contents

Tests	313
T1: Insufficient Tests	313
T2: Use a Coverage Tool!	313
T3: Don't Skip Trivial Tests	313
T4: An Ignored Test Is a Question about an Ambiguity	313
T5: Test Boundary Conditions	314
T6: Exhaustively Test Near Bugs	314
T7: Patterns of Failure Are Revealing	314
T8: Test Coverage Patterns Can Be Revealing	314
T9: Tests Should Be Fast	314

Namen

- **N1 Choose Descriptive Names:** Beschreibende Namen verwenden.

```
public int x() {  
    int q = 0;  
    int z = 0;  
    for (int kk = 0; kk < 10; kk++) {  
        if (l[z] == 10)  
        {  
            q += 10 + (l[z + 1] + l[z + 2]);  
            z += 1;  
            ...  
        }  
    }  
}
```

```
public int score() {  
    int score = 0;  
    int frame = 0;  
    for (int frameNumber = 0; frameNumber < 10; frameNumber++) {  
        if (isStrike(frame)) {  
            score += 10 + nextTwoBallsForStrike(frame);  
            frame += 1;  
            ...  
        }  
    }  
}
```

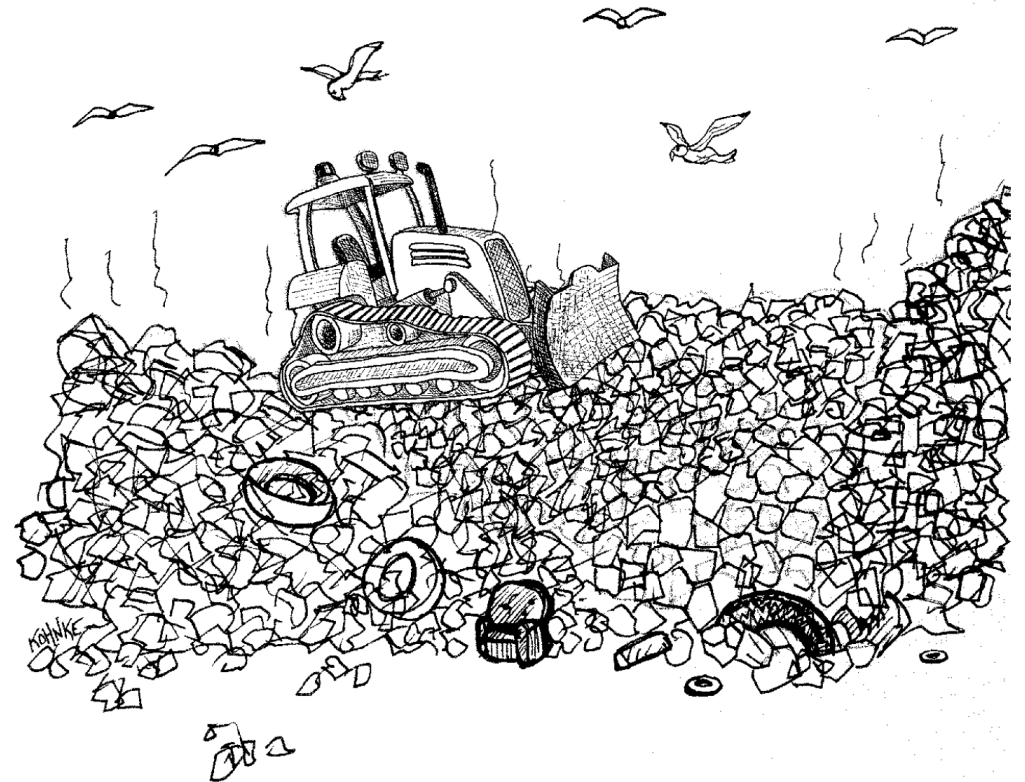
Namen

- **N5 Long Names for Long Scopes:** Wenn Variablen nur in einem kleinen Scope gebraucht werden, können Sie auch kurz sein.

```
private void rollMany(int n, int pins)
{
    for (int i=0; i<n; i++)
        g.roll(pins);
}
```

Refactoring Code Smells

- Refactoring regelmäßig notwendig
- Code Smells entfernen/verbessern
- State-of-the-Art einhalten
- Refactoring erzeugt nicht nur kosten sondern auch Value!



Aufgaben: Refactoring Code Smells

1. Laden Sie sich “salaryApp_smelly.cpp” vom Git und erstellen Sie eine Kopie der Datei zur Bearbeitung
2. Analysieren Sie den Funktionsumfang des Programms
3. Betreiben Sie ein Refactoring für das Programm!

Geben Sie bei jedem Refactoring an welchen Code Smell Sie ersetzt haben (als Kommentar am Anfang des Files)!

Aufgaben: Übungsprojekt



- 1) Überarbeiten Sie ihren bisherigen 4-Gewinnt Code und bauen Sie Clean Code best-Practices ein.
- 2) Erweitern Sie ihr 4-Gewinnt Projekt. Es soll zunächst drei 3 Bots geben:
 - a) RandomBot: legt immer zufällig
 - b) VerticalBot: legt immer in die Spalte die am weitesten links liegt
 - c) HorizontalBot: legt immer in die Spalte mit am wenigsten SteinenÜberlegen Sie sich eine Architektur wie Sie ihr Programm einfach mit neuen Spieler Typen erweitern können.
- 3) Die Menüführung soll ermöglichen wer gegen wen spielt (auch gleiche sollen gegeneinander spielen können).

Kapitel 5: Clean Code

51. Motivation

52. Clean Code – Robert C. Martin



53. Weitere Paradigmen

54. Debugging

Paradigmen

- **KISS – *Keep it Stupid Simple***
 - Gehirn kann nur wenige Tokens gleichzeitig behandeln (RAM Problem)
→ Viele kleine Funktionen sind viel schneller verständlich
- **YAGNI – *You Ain't Gonna Need It***
 - Es sollte nicht zu kompliziert und flexibel programmiert werden wenn es nicht gefordert ist
 - Anforderungen für neue Feature ändern sich sowieso wieder
 - **„Brauche ich das wirklich?“ Wenn die Antwort nicht ein lautes „JA“ ist, dann nicht implementieren!**

The Boy Scout Rule

- „Verlasse den Ort sauberer als du ihn aufgefunden hast.“



Kapitel 5: Clean Code

51. Motivation

52. Clean Code – Robert C. Martin

53. Weitere Paradigmen



54. Debugging

Debugging Werkzeuge

- Breakpoints
- Locals (lokale Variablen)
- Debugausgaben
- Call Stack
- Watch
- Unit Test (kommen noch)



Debugging – Tipps

1. Sprechen Sie mit der Ente! Formulieren Sie das Problem.
2. Reduzieren Sie das Problem
 - a) Hängt das Problem mit falschen Eingabe Werten zusammen?
 - b) Andere Eingabe Werte probieren?
 - c) Kann es Ursachen außerhalb vom Code haben (anderer Compiler etc.)?
3. Grenzen Sie den Code ein
 - a) Kommentieren Sie Teile vom Code aus oder führe nur bestimmte Teile aus um den schuldigen Code einzuschränken
4. Verstehen Sie das Problem (wirklich!)
5. Lösen Sie es Schritt für Schritt
6. Wie kann verhindert werden, dass so etwas nochmal passiert? Unit-Tests, Maßnahmen im Code (const, Abfragen von Input Werten etc.), Architektur, ...
7. Kann das Design verbessert werden um Fehler zu verhindern? → **Clean Code!**



Debugging – Don'ts

1. *„Ich hänge hier seit ner Stunde total fest, aber niemand kennt den Code hier so gut wie ich warum sollte der mir helfen können?“*
2. *„Ich ändere jetzt einfach mal das hier, und gucke was passiert, wenn das nicht passt dann halt dieses und jenes, ...“*
3. *„Ich verstehe zwar nicht genau was hier passiert, aber das wird schon passen, ich finde den Fehler auch so.“*
4. *„Ich habe xy jetzt schon x mal gefragt, ich kann ihn nicht schon wieder fragen.“*

Bugs vermeiden

- *Clean Code*
 - *Test driven development*
 - Garantierte Testabdeckung
 - *Enforced Coding Rules*
 - *Zero-Warning-Policy*
 - **Das Wichtigste:**
Ein nachhaltiges Entwicklungs-Mind-Set der **Entwickler** und der **Manager**
- Verantwortung über guten (clean) Code liegt beim Entwickler

Assert

- Enforcen von bestimmten Bedingungen durch `asserts`
- Führen direkt zu Program-Abbruch
(kann z.B. bei großen Simulationen sehr hilfreich sein)
- Werden in der Regel nur beim Debugging mit kompiliert
(nicht für Produktions Einsatz)

```
47  #include <cassert>
48
49  void doSomething(int input)
50  {
51      // check that input fulfills necessary criteria
52      assert(input < 255);
53      // do something
54  }
```

Assert

Live

Assert

```
01_lecture > 5_Clean_Code > assert > C++ assert_main.cpp > main()
1  #include <iostream>
2  #include <cassert>
3
4  // Funktion zur Division von zwei Zahlen mit assert zur Überprüfung des Divisors
5  double divide(double numerator, double denominator) {
6      assert(denominator != 0 && "Denominator must not be zero");
7      return numerator / denominator;
8  }
9
10 // Funktion zur Demonstration der Verwendung von asserts in einem Simulation
11 void runSimulation(int numberOfIterations) {
12     assert(numberOfIterations > 0 && "Number of iterations must be greater than zero");
13
14     for (int i = 0; i < numberOfIterations; ++i) {
15         // Beispiel: Berechnung in der Simulation
16         double result = divide(100.0, i + 1); // i+1, um Division durch 0 zu vermeiden
17         std::cout << "Iteration " << i + 1 << ": Result = " << result << std::endl;
18     }
19 }
20
21 int main() {
22     // Setzen der Anzahl der Iterationen
23     int numberOfIterations = 0;
24
25     // Aufruf der Simulationsfunktion
26     runSimulation(numberOfIterations);
27
28     return 0;
29 }
30
```